

LED Checkers

Project Title:

Smart Checkers Board with Hall Effect Sensors and Real-Time LED Display

Course:

Engineering Sciences 50 – Introduction to Electrical Engineering

Team Members:

Muhammed Khan, Federico Garcia Alzate

Date:

May 1, 2025

Abstract

For our final project in ES50, we designed and built a smart checkerboard that detects the presence of magnetic checker pieces on the board and shows their positions in real-time in a 16x16 LED matrix. We implemented 32 digital Hall effect sensors (DRV5032DU) along the 32 black squares on the 8x8 checkers board. These sensors detect the presence of magnets placed in the bottom of checker pieces. Then, each row of sensors was connected to one of the eight SN74HC165N shift registers we used. These shift registers were connected to the Arduino Uno R3, which received serial numbers with the sensor data, then processed it, and made a corresponding 2x2 square on the LED matrix light up to represent the state of the game board. The final checkers board was elevated using 3D-printed legs designed in Solidworks. The circuits were hidden underneath a 3D-printed platform, which was also designed using Solidworks. In our initial goal, we aimed to incorporate a validation and suggestion system through the use of physical buttons. Still, we prioritized stable real-time tracking and could not complete that extension, particularly because of the limitations of the sensors. The system is fully functional for position tracking in real-time and provides a foundation for future implementations of in-game logic and interactivity.

Introduction

We created a project that integrates the physical gameplay with digital feedback to offer a more visually appealing interactive experience. We aimed to incorporate an embedded system in the checker's board, which detects player moves and provides visual help. By doing this, we aimed to enhance the classic analog game with digital features while keeping its hands-on feel.

Checkers is an excellent choice for this project because of its simple board format, precise movements, and a limited number of legal moves. Unlike touchscreen games, our physical design allows for face-to-face gameplay and provides immediate visual feedback on the game state using an attractive LED matrix.

Throughout the project, we applied many electrical engineering topics we learned in class, such as shift register communication, neopixel control, and analog circuits. We combined multiple systems—sensing, processing, display, and the physical setup—into one project. The final product is a working, real-time checker tracker perfect for classroom use, public displays, or casual gaming.

Design and Functionality

The final product is a fully functional smart checkers board that detects the presence of magnetic game pieces on all 32 dark squares. For this, an array of DRV5032DU Hall effect sensors was placed under each dark square, making the system continuously monitor the board squares for the place of the pieces and update a 16x16 WS2812B NeoPixel LED matrix in real-time. Each playable square is represented by a 2x2 LED block, giving a clear and immediate view of the current piece layout.

The electronics are arranged so each board row is connected to its own SN74HC165N shift register. This setup allows for efficient handling of sensor inputs as the shift register will send a serial number with the sensor information for each row. The board is raised using custom 3D-printed legs, and the wiring is beneath a 3D-printed platform.

The code uploaded to the Arduino allows LED mapping, coordinate changes, and real-time game state tracking.

Hardware Design

Sensors

We used 32 DRV5032DU Hall effect sensors — one for each dark square of the board. These digital sensors have a cutoff depending on the magnetic field, making them output LOW when a magnet is present and HIGH when it's not. This makes them great for working with shift registers and interpreting presence as a simple binary signal.

Each sensor was soldered to insulated wires and glued to the underside of the board with hot glue. We picked the DRV5032DU because it is small and straightforward, mainly because the price fell within our budget. The magnets in the checker's pieces are strong neodymium discs, small enough to be discreet but powerful enough to activate the sensors effectively.

Shift Registers

Since the Arduino Uno R3 has a limited number of input pins, we used eight SN74HC165N shift registers to read 32 sensors using just a few pins. Each shift register manages four sensor inputs, adding up to a total of 32.

The registers aren't connected in a chain. Each one has its own QH (data) pin connected to a different digital input on the Arduino, using pins 13 down to 6. They all share the same clock and latch pins. This setup made testing and debugging easier and allows us to control each row independently.

Each row of sensors connects to a shift register, and the four active bits in a shift register correspond to the black squares in that row (like a1, c1, e1, g1 for row 1). We made sure all the wiring was organized and tested step-by-step during development.

LED Matrix

We used a 16x16 WS2812B NeoPixel matrix to display the board state. Each square on our 8x8 board is shown as a 2x2 block on the LED matrix, allowing easy control over each piece.

Mapping the board to the matrix was tricky because of the zigzag wiring and because the starting point (0,0) on the matrix is at the bottom-left of our setup. We created a custom mapping function to convert board coordinates to the right LED positions. Two separate 4.5V battery packs power the matrix. The batteries are connected to the matrix through an SPDT On-On switch, allowing them to turn on and off the matrix.

Mechanical Design

We created some 3D-printed custom parts for the mechanical design to support and cover the electronics under the board. This included four strong corner legs to lift the board, two side legs to connect the two sections of the board, and a platform that hides the wiring and supports the LED matrix, ensuring they align properly beneath their respective squares. We used PLA for all the parts because it's durable and easy to work with. The design has enough space for wiring and ventilation while still looking neat. The electronics are completely hidden once everything is put together, so the focus is on the gameplay surface.

Software Design

Overview

The software performs the following loop:

1. Reads 8 shift registers using `shiftIn()` after a shared `loadPin` latch.

2. For each of the 8 bits per row, it checks whether the bit corresponds to a valid board square.
3. If so, it looks up the (x, y) LED coordinates for that square using the `shiftToXY[row][bit]` matrix.
4. If a sensor reads LOW (magnet present), lights up the 2x2 block in **green**.
5. If HIGH (no magnet), lights it up in **yellow**.
6. Writes the board state to the `board[8][8]` array.
7. Calls `FastLED.show()` to refresh the entire LED matrix.

Mapping

We created a matrix:

```
cpp
CopyEdit
const int shiftToXY[8][8][2];
```

Where each index `[row][bit]` contains the corresponding LED matrix coordinates (x, y). Invalid positions (white squares) are set to `{-1, -1}` and skipped.

The `XY()` function takes the (x, y) and translates it into the correct NeoPixel index, flipping coordinates appropriately for the serpentine wiring.

Draw 2x2()

This helper function lights up a 2x2 block starting from the bottom-right corner. It loops over (dx, dy) and calculates the LED index for each of the four pixels to light.

Features Implemented

The project brings the checkers board to life by providing more cool features. At its heart, it tracks the checkers in real-time using magnets in the pieces and digital Hall effect sensors located under each square, allowing it to know when a piece is on the board instantly. In contrast, the LED matrix shows this information visually by implementing green lights to highlight

squares with a piece and yellow for empty ones. A solid coordinate conversion system helps translate the sensor data into the correct LED positions, keeping everything accurate. The physical structure features a sturdy 3D-printed platform and enclosure, making it look neat and professional. Furthermore, a flexible board state array keeps track of the game's logic and lays the groundwork for future features like move validation, capture detection, and digital game logging.

Limitations and Challenges

1. Move Suggestion Not Finished

We planned to use two physical buttons to toggle valid move highlighting for red and black pieces. This would have required scanning the `board[ ][ ]` array for legal diagonal moves and lighting up empty targets in a third color. While the physical buttons were wired and software was scaffolded, we did not complete the logic before the deadline.

2. Sensor Uniformity

Digital Hall sensors do not distinguish between types of magnets. This limits us from detecting which color piece is on a square. With the implementation of analog Hall effect sensors, we would have been able to detect different colors and exceptional cases, such as distinguishing a king piece from a regular piece, by using magnets with a variety of strengths.

3. LED Current Draw

While our matrix worked fine at low brightness, a full-power setup would require a high-current power supply or external 5V source with a greater current, instead of two 4.5V battery packs in parallel. We worked around this by lowering the brightness to 10/255.

Future Work

There are multiple ways to improve this project for future versions. The first change is to use analog Hall effect sensors instead of digital ones. This would help the system recognize different magnetic field strengths, allowing us to distinguish between black and red pieces and exceptional cases based on the type of magnets. The second change is adding move logic. By now, the board only tracks where pieces are, but if we add the ability to suggest and check legal moves, it would make the game much more enjoyable and help people learn checkers through the game.

Bill of Materials (BOM)

Conclusion

We built a working interactive checkers board that immediately shows the change of place of the pieces on it. The project addressed several challenges: soldering sensors, creating and building circuits using shift registers, reading data from these registers, aligning the LED, and making physical components to improve the aesthetics. The system performs as expected, is easy to understand, and can be modified for multiple games. Although we didn't finish the validation move function and outputting possible moves, this project shows the combination of electrical engineering, software, and mechanical design knowledge.

Acknowledgments

We thank our course staff, including instructors and TAs, for their support and guidance. The class's lab progression gave us all the tools we needed to build this from the ground up.

<https://github.com/mkhan-harvard/LEDCheckers>